

LAPORAN TUGAS BESAR ALGORITMA DAN PEMROGRAMAN
OPTIMASI ALUR KERJA CI/CD
MENGGUNAKAN MULTISTAGE GRAPH



Mata Kuliah:

Algoritma dan Pemrograman

Anggota Kelompok:

Cornelius Fransinatra Wijaya	21120124140141
Ashar Firdaus	21120124130062
Andre Jonathan Tampubolon	21120124130050

BAB 1

PENDAHULUAN

1.1 Latar Belakang Studi Kasus

Dalam rekayasa perangkat lunak modern, otomatisasi alur kerja melalui Continuous Integration & Continuous Deployment (CI/CD) menjadi fondasi utama untuk memastikan pengiriman kode yang cepat, aman, dan berkualitas tinggi. Alur kerja CI/CD pada umumnya terdiri dari beberapa tahapan sekuensial. Dalam studi kasus ini, proses otomatisasi dibagi menjadi empat tahap utama: Code Analysis (T1), Automated Testing (T2), Security Scanning (T3), dan Deployment (T4). Di setiap tahapan, tim pengembang dihadapkan pada berbagai pilihan layanan (tools atau worker) yang memiliki karakteristik performa berbeda, ditandai oleh variasi biaya operasional dan waktu eksekusi.

Pilihan layanan pada satu tahap sering kali memengaruhi efisiensi tahap berikutnya karena adanya ketergantungan antar-alat. Oleh karena itu, penentuan kombinasi layanan yang optimal dari awal hingga akhir tidak dapat diselesaikan dengan pendekatan rakus (greedy approach) biasa, melainkan harus dimodelkan sebagai struktur graf berlapis atau disebut sebagai Multistage Graph.

1.2 Tujuan Optimasi

Tujuan utama dari tugas besar ini adalah menemukan rute atau kombinasi layanan dari simpul asal (S) menuju simpul akhir (E) yang menghasilkan efisiensi tertinggi. Efisiensi tersebut diukur dengan meminimalkan total Bobot (W). Formula penentuan bobot pada setiap edge didasarkan pada kombinasi linier antara dua variabel utama, yaitu Biaya dan Waktu, dengan bobot pengaruh masing-masing sebesar 60% dan 40%. Rumus matematis yang digunakan adalah:

$$\text{Bobot (W)} = (\text{Biaya} \times 0,6) + (\text{Waktu} \times 0,4)$$

Dengan meminimalkan fungsi tujuan tersebut, perusahaan dapat menghemat pengeluaran finansial operasional sekaligus menjaga kecepatan siklus rilis perangkat lunak tetap berada pada titik optimal.

BAB 2

VISUALISASI DAN LANDASAN TEORI

2.1 Deskripsi Naratif Struktur Graf

Struktur alur kerja dimodelkan sebagai Multistage Graph berarah yang terdiri dari simpul asal S, simpul tujuan akhir E, serta simpul-simpul antara yang terbagi ke dalam 4 tahapan (*stages*). Hubungan konektivitas dan bobot antar-simpul dirinci sebagai berikut:

Tahap 1 - Code Analysis (T1)

Simpul asal S terhubung ke dua pilihan simpul, yaitu Node A1 (Tools standar) dengan bobot $W = 15$, dan Node A2 (Tools premium) dengan bobot $W = 25$.

Tahap 2 - Automated Testing (T2)

Node A1 terhubung ke B1 (Unit test lokal) dengan $W = 35$ dan ke B2 (Integration Testing) dengan $W = 45$. Sementara itu, Node A2 terhubung ke B1 dengan $W = 20$ dan ke B2 dengan $W = 30$.

Tahap 3 - Security Scanning (T3)

Node B1 terhubung ke C1 (Basic scan) dengan $W = 25$ dan ke C2 (Advanced Pen-Test) dengan $W = 55$. Node B2 terhubung ke C1 dengan $W = 40$ dan ke C2 dengan $W = 30$.

Tahap 4 - Deployment (T4)

Node C1 terhubung ke simpul akhir E (Provider X) dengan $W = 20$, sedangkan Node C2 terhubung ke E (Provider Y) dengan $W = 15$.

Berikut adalah tabel representasi seluruh keterhubungan *edge* dan bobot di dalam graf:

Simpul Asal (From)	Simpul Tujuan (To)	Keterangan Layanan	Bobot (W)
S	A1	T1 - Tools Standar	15
S	A2	T1 - Tools Premium	25

Simpul Asal (From)	Simpul Tujuan (To)	Keterangan Layanan	Bobot (W)
A1	B1	T2 - Unit Test Lokal	35
A1	B2	T2 - Integration Testing	45
A2	B1	T2 - Unit Test Lokal	20
A2	B2	T2 - Integration Testing	30
B1	C1	T3 - Basic Scan	25
B1	C2	T3 - Advanced Pen-Test	55
B2	C1	T3 - Basic Scan	40
B2	C2	T3 - Advanced Pen-Test	30
C1	E	T4 - Provider X	20
C2	E	T4 - Provider Y	15

2.2 Hubungan Fungsi $F(\text{stage}, \text{mode})$ Berbasis Pemrograman Dinamis

Prinsip *Dynamic Programming* memecah masalah optimasi graf ini menjadi sub-masalah berlapis yang saling tumpang tindih (*overlapping subproblems*). Kita dapat mendefinisikan fungsi secara *Backward* (bergerak mundur dari simpul tujuan E ke simpul asal S) untuk menentukan nilai bobot minimum.

Misalkan k menyatakan total tahapan dalam graf ($k = 4$ tahap transisi, atau terdiri dari rentang *stage* $i = 1$ hingga 5 simpul tingkat). Kita definisikan fungsi $F(i, j)$ sebagai biaya rute minimum dari simpul j pada tahapan/stage i menuju simpul akhir E. Formula rekurensinya adalah:

$$F(i, j) = \min_r \{c(j, r) + F(i + 1, r)\}$$

Di mana:

1. i melambangkan indeks tahapan (*stage*) saat ini ($i \in \{1, 2, 3, 4\}$).
2. j melambangkan identitas simpul (*mode* / nama alat) pada tahapan i .
3. r melambangkan seluruh simpul tetangga pada tahapan berikutnya ($i + 1$) yang terhubung langsung dari simpul j .
4. $c(j, r)$ melambangkan bobot langsung (*edge weight*) dari simpul j ke simpul r .

Kondisi Batas (*Base Case*): Untuk simpul tujuan akhir pada tahap akhir, nilai fungsi didefinisikan sebagai $F(5, E) = 0$.

BAB 3

IMPLEMENTASI PROGRAM

3.1 Desain Struktur Data

Program diimplementasikan menggunakan bahasa pemrograman C++ dengan pendekatan berbasis objek yang berfokus pada manajemen memori secara eksplisit melalui pointer guna mereplikasi perilaku referensi objek di Java. Tiga komponen struktur data utama meliputi:

1. `Struct Node`: Merepresentasikan simpul layanan yang menyimpan atribut `name` (string), `ds` (distance dari source, diinisialisasi dengan nilai tak hingga atau `INT_MAX`), `visited` (boolean), dan sebuah pointer `parent` yang menunjuk ke objek `Node` sebelumnya untuk rekonstruksi jalur (*path tracking*).
2. `Struct Edge`: Merepresentasikan jalur penghubung antar-layanan, yang menyimpan pointer `node1` (asal), `node2` (tujuan), dan integer `w` (bobot kombinasi biaya dan waktu).
3. `Custom Comparator (CompareNode)`: Sebuah objek fungsi (*functor*) yang membebani operator `()` untuk membalikkan urutan elemen pada antrean prioritas (`std::priority_queue`) agar bertindak sebagai Min-Heap berdasarkan nilai `ds` terkecil.

3.2 Alur Logika Algoritma Dijkstra

Logika utama dijalankan di dalam metode `runDijkstra()`. Alur algoritma dapat dijabarkan dalam langkah-langkah terstruktur berikut:

1. **Inisialisasi**: Jarak simpul awal `S` diatur menjadi 0 (`s->ds = 0`) dan pointernya dimasukkan ke dalam `priority_queue`. Semua simpul lainnya diinisialisasi dengan nilai tak hingga atau `INT_MAX`.
2. **Loop Utama (Ekstraksi)**: Selama antrean prioritas tidak kosong, simpul dengan nilai `ds` terkecil diekstrak sebagai simpul `current`, kemudian status `current->visited` diubah menjadi `true`.
3. **Pencarian Tetangga dan Relaksasi**: Program memanggil fungsi pembantu `getAdjacentNodes(current)` untuk mendapatkan daftar simpul tujuan yang terhubung.

Untuk setiap simpul `next` (tetangga), program mencari bobot spesifiknya melalui fungsi `getWeight(current, next)`. Jika ditemukan jalur yang lebih pendek ($next.ds > current.ds + weight$), maka dilakukan pembaruan jarak (`ds`), *pointer parent* diarahkan ke `current`, dan simpul `next` dimasukkan ke dalam antrean prioritas untuk dievaluasi ulang.

4. Cetak Hasil: Jalur dirunut mundur menggunakan fungsi rekursif `printPath` dari simpul tujuan ke simpul asal berdasarkan jejak pointer `parent`.

BAB 4

ANALISIS KOMPLEKSITAS

4.1 Analisis Kasus Terburuk (Worst-Case Analysis)

Karakteristik utama dari implementasi kode program yang dibuat terletak pada cara graf disimpan dan diakses. Program tidak menggunakan struktur Adjacency List tradisional, melainkan menggunakan struktur Edge List global berupa `std::vector<Edge*> graph`. Akibatnya, operasi penelusuran struktur graf harus diproses secara sekuensial linear.

Dua fungsi pembantu eksternal yang dipanggil di dalam loop utama memiliki analisis individual sebagai berikut:

1. `getAdjacentNodes(Node* current)`: Fungsi ini melakukan iterasi melewati seluruh koleksi edge di dalam vektor graf dari awal hingga akhir guna mencocokkan nama simpul asal. Operasi ini membutuhkan waktu sebanding dengan total jumlah edge di dalam graf, yaitu sebesar $\mathcal{O}(E)$.
2. `getWeight(Node* a, Node* b)`: Serupa dengan fungsi sebelumnya, untuk menemukan bobot numerik dari sebuah relasi, fungsi ini kembali melakukan pencarian linear melewati seluruh elemen vektor graf berukuran E. Kompleksitas waktu untuk satu kali pemanggilan adalah $\mathcal{O}(E)$.

4.2 Pembuktian Notasi Big O

Untuk membuktikan kompleksitas total algoritma dalam notasi Big O, kita harus menjabarkan total operasi yang dilakukan berdasarkan interaksi loop bersarang (*nested loops*) yang terbentuk:

Misalkan K adalah total frekuensi operasi ekstraksi (`pop`) elemen dari `priority_queue` di dalam loop utama `while (!pq.empty())`. Pada algoritma Dijkstra tanpa pencegahan eksplisit elemen duplikat di awal loop, sebuah simpul dapat masuk ke dalam antrian berkali-kali setiap kali terjadi proses relaksasi yang berhasil. Namun, karena struktur graf ini berbentuk Multistage Directed Acyclic Graph (DAG), jumlah maksimum ekstraksi dibatasi oleh derajat masuk simpul.

Untuk setiap iterasi dari K kali perulangan luar tersebut, eksekusi program di dalamnya adalah sebagai berikut:

1. Memanggil `getAdjacentNodes(current)` → Membutuhkan waktu $\mathcal{O}(E)$.
2. Melakukan perulangan untuk mengevaluasi setiap simpul tetangga yang dikembalikan. Jumlah tetangga maksimum dari sebuah simpul dibatasi oleh jumlah simpul pada tahap berikutnya, yang secara umum bernilai lebih kecil atau sama dengan total simpul V: $\deg^+(u) \leq V$.
3. Di dalam perulangan tetangga tersebut, fungsi `getWeight(current, next)` dipanggil → Membutuhkan waktu $\mathcal{O}(E)$ untuk setiap tetangga.

Dengan demikian, total kompleksitas waktu di dalam satu kali iterasi loop utama ditentukan oleh penjumlahan operasi linear search dan perulangan internal:

$$T_{internal} = \mathcal{O}(E) + \deg^+(current) \times \mathcal{O}(E)$$

Karena dalam kondisi terburuk (*worst-case*), derajat keluar simpul dapat mendekati orde jumlah simpul ($\deg^+(current) \approx V$), maka kompleksitas internal per iterasi disederhanakan menjadi $\mathcal{O}(V \cdot E)$. Jika loop utama dieksekusi sebanyak K kali, maka total waktu eksekusi teoritis dari keseluruhan algoritma dalam Notasi Big O adalah:

$$\text{Total Worst-Case Complexity} = \mathcal{O}(K \times V \times E)$$

Pada graf studi kasus terikat ini dengan jumlah komponen konstan ($V = 8$ dan $E = 12$), program berjalan sangat cepat. Namun secara teoretis untuk skala besar, skema ini kurang efisien dibanding struktur Adjacency List standar yang mampu memangkas waktu pencarian tetangga menjadi $\mathcal{O}(1)$ dan pencarian bobot terikat derajat simpul.

BAB 5

KESIMPULAN

Berdasarkan pemodelan studi kasus alur kerja otomatisasi CI/CD menggunakan *Multistage Graph*, implementasi program algoritma Dijkstra dalam bahasa C++ telah berhasil menemukan kombinasi rute layanan yang paling optimal dengan total bobot seminimal mungkin. Dari hasil perhitungan visual, eksekusi program memberikan solusi rute terbaik yaitu $S \rightarrow A2 \rightarrow B1 \rightarrow C1 \rightarrow E$ dengan akumulasi nilai total **Bobot Minimal = 90**.

Meskipun struktur data *Edge List* linear global memicu timbulnya perulangan bersarang (*nested loop*) di dalam fungsi pembantu pembentuk graf sehingga menciptakan kompleksitas kasus terburuk sebesar $O(K \cdot V \cdot E)$, akurasi hasil pencarian jalur terpendek tetap terjaga sepenuhnya. Rekomendasi pengembangan lebih lanjut untuk peningkatan performa pada skala graf yang lebih besar adalah dengan melakukan migrasi representasi graf ke dalam bentuk daftar ketetanggaan (*Adjacency List*).

LAMPIRAN

[1] Ponk2System, "CICD Multistage Optimizer," 2026. [Daring]. Tersedia:
<https://github.com/Ponk2System/cicd-multistage-optimizer>